

BuMoChi はじめに

BuMoChi

概要

このガイドでは、小さなテストを順番に行いながら BuMoChi を紹介します。各手順では、前の手順で動作を確認したパイプラインと作成した素材を引き続き使用します。

ソフトウェアをまだインストールしていない場合は、[インストール](#)に従ってください。

システムをセットアップしてアニメーションデータを受信し、Godot でアニメーションを実行する方法については、[BuMoChi フルセットアップ手順](#)を参照してください。

最初の 5 つの演習では、現在のデフォルト設定を使用します。

コンポーネント	設定
XR-Animator VMC 出力	127.0.0.1:39537
BunrakuOSCEncoder 入力	39537
Bmc 入力	57130
BunrakuOSCDecoder 入力	39538
Ishidomaru の Godot VMC 入力	39539
Godot プロジェクト	Seed_2_Ishidomaru_C

ターミナルコマンドは BuMoChi リポジトリのルートから実行してください。SuperCollider コードは、括弧で囲まれたブロックまたは 1 つの文を単位として評価します。

XR-Animator から Ishidomaru を動かす

Web カメラの前で動いてください。Godot 内の Ishidomaru がその動きに追従するはずですが、

Bmc モニターで、動的な **received** カウントが増加していることを確認します。エンコーダーのターミナルにも、受信した VMC データと Bmc に送信したフレームが表示されます。

アバターが動かない場合は、下流から上流の順にパイプラインを確認します。

1. Godot が **Seed_2_Ishidomaru_C** を実行し、**39539** で待ち受けている。
2. デコーダーが **39538** で待ち受け、送信先ポート **39539** を許可している。
3. Bmc が **57130** で待ち受け、デコーダーへ転送している。
4. エンコーダーが **39537** で待ち受け、**57130** の Bmc に送信している。
5. XR-Animator が VMC を **127.0.0.1:39537** に送信している。

さらに詳しく診断するには、[Port Number Setup](#)を参照してください。

アルゴリズムで生成したクリップから Ishidomaru を動かす

この例では、最後に受信した有効なライブポーズをモデル互換の参照として取得し、腰が左右へゆっくり動く 4 秒間のクリップを生成します。有効な受信ポーズを使うことで、Ishidomaru のボーン比率と VMC 座標規則が保たれます。

まず、楽な直立姿勢を取り、Ishidomaru が表示されていることを確認します。その後、次を評価します。

```
(  
var baseFrame = Bmc.defaultAvatar.currentFrame;  
var frameRate = 60;  
var duration = 4.0;
```

```

var frameCount = (frameRate * duration).asInteger;
var entries;
var clip;

if(baseFrame.isNil) {
    Error("No live Ishidomaru frame has arrived yet").throw;
};

entries = Array.fill(frameCount, { |index|
    var time = index / frameRate;
    var pose = baseFrame.pose.copy;
    var hips = pose.at(\Hips).asArray;
    var phase = time * 2pi * 0.25;

    hips[0] = hips[0] + (sin(phase) * 0.10);
    pose.put(\Hips, hips);

    [
        time,
        BmcFrame(
            "Ishidomaru",
            "algorithmic-sway",
            index,
            time,
            pose
        ).asOSCMessage
    ]
});

clip = BmcAnimationClip(entries, (
    generator: \sine,
    bodyPart: \Hips,
    duration: duration,
    frameRate: frameRate
));

Bmc.library.add(\algorithmicSway, clip);
Bmc.saveClipScd(\algorithmicSway);
)

```

ライブストリームと再生が競合しないように、XR-Animator の VMC 出力を一時的に無効にします。その後、生成したクリップを再生します。

```

Bmc.loop(true);
Bmc.play(\algorithmicSway);

```

Ishidomaru がゆっくりした左右運動を繰り返します。次のコードで停止します。

```

Bmc.stopPlayback;
Bmc.loop(false);

```

録画の演習へ進む前に、XR-Animator の VMC 出力を再び有効にしてください。

クリップを録画する

XR-Animator のライブアニメーションが動作している状態で、名前を付けて録画を開始します。

```

Bmc.record(\take1, "Ishidomaru", "getting-started-xr");

```

数秒間動いた後、録画を停止します。

```

~take1 = Bmc.stopRecording;

```

結果を確認します。

```

~take1.size;

```

```
~take1.duration;  
Bmc.listClips;  
BmcClipLibrary.defaultDirectory;
```

take1 はメモリー内に保持されます。また、SCD がデフォルトの録画形式であるため、**BmcClipLibrary.defaultDirectory** に **take1.scd** として保存されます。

返されたクリップのフレーム数がゼロの場合は、録画中に Bmc モニターの **received** カウントが増加していたか、エンコーダーが上記と同じアバター名とソース文字列を使用していたか確認してください。

クリップを再生する

ライブデータが録画と競合しないように、XR-Animator の VMC 出力を無効にします。クリップを再生します。

```
Bmc.play(\take1);
```

基本的なトランスポート操作を試します。

```
Bmc.pause;  
Bmc.resume;  
Bmc.stopPlayback;
```

半分の速度でループ再生します。

```
Bmc.rate(0.5);  
Bmc.loop(true);  
Bmc.play(\take1);
```

終了後はデフォルト値に戻します。

```
Bmc.stopPlayback;  
Bmc.rate(1.0);  
Bmc.loop(false);
```

クラスライブラリを再コンパイルした後でもディスク上のコピーを復元できることを確認するには、再コンパイルしてから明示的に読み込みます。

```
Bmc.loadClipScd(  
    BmcClipLibrary.defaultDirectory +/+ "take1.scd",  
    \take1  
);  
Bmc.play(\take1);
```

1 体のアバター上で 2 つのクリップを合成する

この演習では **take1** を基礎モーションとして使用し、両腕のデータを 2 つ目の録画から取得します。

XR-Animator の VMC 出力を再び有効にし、特徴的な腕の動きを含む 2 つ目のテイクを録画します。

```
Bmc.record(\armTake, "Ishidomaru", "getting-started-xr");
```

数秒間腕を動かした後、録画を停止します。

```
Bmc.stopRecording;
```

タイミングと下半身には **take1** を使用し、左右の腕には **armTake** を使用する新しいクリップを作成します。

```
Bmc.combineClips(  
    \take1,  
    \armTake,  
    \arms,  
    \combinedTake  
);  
Bmc.saveClipScd(\combinedTake);
```

XR-Animator の VMC 出力を無効にし、結果を再生します。

```
Bmc.play(\combinedTake);
```

合成クリップの長さは **take1** と同じです。腕のデータが置き換えられるのは両方のクリップに存在するフレーム数までで、それ以降の基礎フレームは変更されません。

クリップと XR-Animator から 2 体のアバターを動かす

この演習では、2 体を均一に扱うプロジェクトを使用します。Ishidomaru は XR-Animator のライブソースに追従し、Mother は保存済みクリップを独立して再生します。

1. Godot プロジェクトを変更する

現在の Godot シーンを停止し、次を開いて実行します。

AppsAndCode/GodotProjects/Seed_4_Mother_Ishidomaru_E/project.godot

このプロジェクトでは次のポートを使用します。

アバター	VMC ポート
Mother	39539
Ishidomaru	39540

2. 2 体のアバター用にデコーダーを再起動する

Control-C で既存のデコーダーを停止し、次を実行します。

```
python3 PipelineApplications/Bunraku0SCDecoder.py \  
--listen-port 39538 \  
--allow-target-port 39539 \  
--allow-target-port 39540 \  
--verbose
```

3. Bmc に 2 つのアバター出力を登録する

次を評価します。

```
(  
~ishidomaru = Bmc.avatar(\Ishidomaru);  
~ishidomaru.vmcPort_(39540);  
  
~mother = Bmc.avatar(\Mother);  
if(~mother.isNil) {  
    ~mother = Bmc.addAvatar(\Mother, "Mother");  
};  
~mother.vmcPort_(39539);  
)
```

4. Mother でクリップを再生する

Mother 用の独立したプレイヤーを作成します。これは Bmc の単一の簡易プレイヤーとは別のため、Ishidomaru のライブ入力と Mother のクリップ再生を同時に行えます。

```
~motherPlayer = BmcClipPlayer(Bmc.clip(\combinedTake), ~mother);  
~motherPlayer.loop_(true);  
~motherPlayer.play;
```

5. Ishidomaru のライブ入力を復元する

XR-Animator の VMC 出力を再び有効にします。エンコーダーでは引き続き次の値を使用します。

Avatar: Ishidomaru
Source: getting-started-xr

Ishidomaru が Web カメラに追従し、Mother が **combinedTake** を繰り返すはずです。

Mother の独立した再生を停止します。

~motherPlayer.stop;

この例では、2つの独立したモーション経路を構築しました。将来、セッション定義とフィギュア合成システムによって、複数の割り当てを保存して連携させる、より高水準な方法が提供されます。

終了する

再生と Bmc を停止します。

```
if(~motherPlayer.notNull) { ~motherPlayer.stop };  
Bmc.stopPlayback;  
Bmc.stop;
```

その後、XR-Animator の VMC 出力を無効にし、**Control-C** でエンコーダーとデコーダーを停止して、Godot シーンを停止します。